

Advanced RAG

Retrieval-Augmented Generation

Deep Dive Technical Reference

For AI / ML Practitioners & Engineers

Topics Covered

- * **Retrieval Optimization**

Top-K Tuning * Reranking * Cross-Encoder Reranking

- * **Query Techniques**

Query Rewriting * Multi-Query Retrieval * Query Expansion

- * **Graph RAG [Important]**

Knowledge Graph Retrieval * Entity Relationships * Neo4j

- * **Multi-Modal RAG**

Text + PDF + Image * Structured + Unstructured Data

Table of Contents

01 Retrieval Optimization	3
1.1 Top-K Tuning	3
1.2 Reranking Overview	4
1.3 Cross-Encoder Reranking	5
02 Query Techniques	6
2.1 Query Rewriting	6
2.2 Multi-Query Retrieval	7
2.3 Query Expansion	8
03 Graph RAG (Important)	9
3.1 Knowledge Graph-Based Retrieval	9
3.2 Entity Relationships	1 1
04 Multi-Modal RAG	1 3
4.1 Text + PDF + Image Ingestion	1 3
4.2 Structured vs Unstructured Data	1 5

01 Retrieval Optimization

Retrieval optimization is the process of improving how relevant documents are fetched from a vector store or index. It encompasses controlling the number of results returned, improving their relevance ordering, and applying more accurate but computationally expensive models to re-score candidates after initial retrieval.

1.1 Top-K Tuning

The **Top-K parameter** controls how many candidate chunks are retrieved from the vector store during similarity search. It is one of the most impactful and frequently overlooked tuning levers in a RAG pipeline.

Why Top-K Matters

- **Too small (K=1–2):** High precision but low recall — the LLM misses critical context, leading to incomplete or hallucinated answers.
- **Too large (K=20–50):** High recall but low precision — the LLM's context window floods with noise, degrading answer quality and increasing latency/cost.
- **Optimal range:** K=4 to K=10 works well for most production systems, but always validate empirically with your domain data.

Dynamic Top-K Strategies

Instead of a fixed K, advanced systems adapt K dynamically based on the query complexity, confidence scores of retrieved results, or domain-specific heuristics.

Strategy	Description	Use Case
Score-threshold K	Retrieve until cosine sim drops below θ (e.g. 0.75)	High-precision fact lookup
Query complexity K	Simple queries $\rightarrow$ K=3; complex multi-hop $\rightarrow$ K=10	Mixed workloads
MMR-based K	Max Marginal Relevance to reduce redundancy	Long-doc summarization
Adaptive K (ML)	Train a classifier to predict optimal K per query	Enterprise search

```
# LangChain example: dynamic Top-K with score threshold
retriever = vectorstore.as_retriever(
    search_type='similarity_score_threshold',
    search_kwargs={'score_threshold': 0.75, 'k': 8}
)
```

1.2 Reranking Overview

Initial retrieval using dense embeddings (e.g., FAISS, Chroma) optimizes for **speed** via approximate nearest-neighbor search. This produces a candidate set that is *good but not perfect* in relevance ordering. **Reranking** applies a second, more accurate scoring model to reorder these candidates before passing them to the LLM.

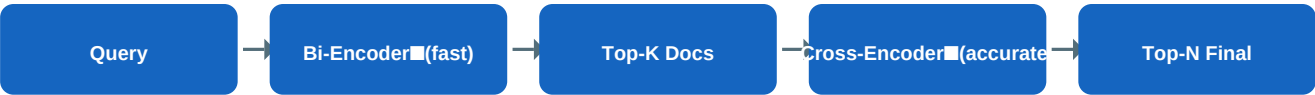


Figure 1.1 — Two-Stage Retrieval Pipeline

Stage 1: Bi-Encoder (ANN)	Stage 2: Cross-Encoder (Rerank)
• Encodes query and docs separately • Pre-computes doc embeddings offline • Sub-millisecond lookup via FAISS/HNSW • Scores: cosine / dot-product similarity • Retrieves K=20–100 candidates quickly	• Jointly encodes query + doc together • Captures cross-attention interactions • More accurate relevance scoring • Runs on top-K candidates only (not full corpus) • Returns reranked top-N (N < K) to LLM

1.3 Cross-Encoder Reranking

Cross-encoder models are the backbone of state-of-the-art reranking. Unlike bi-encoders that create independent embeddings, cross-encoders process the **concatenated** [CLS] query + [SEP] document pair through a transformer encoder (e.g., BERT, RoBERTa) to output a single relevance score.

Popular Cross-Encoder Models

Model	Base	Notes
ms-marco-MiniLM-L-6-v2	MiniLM	Fastest; good for latency-critical apps
ms-marco-electra-base	ELECTRA	High accuracy, moderate speed
bge-reranker-large	BGE/BERT	State-of-art on MTEB benchmark
Cohere Rerank API	Proprietary	Managed; enterprise-grade, multilingual
Jina Reranker v2	Jina-BERT	Long-context support up to 8192 tokens

Implementation with sentence-transformers

```
from sentence_transformers import CrossEncoder

model = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')

# candidate_docs: list of (query, doc_text) pairs
pairs = [(query, doc) for doc in candidate_docs]
scores = model.predict(pairs)

# Sort candidates by reranker score (descending)
ranked = sorted(zip(scores, candidate_docs), reverse=True)
top_n = [doc for _, doc in ranked[:5]] # Take top-5 for LLM
```

LangChain + Cohere Rerank Integration

```
from langchain.retrievers.contextual_compression import ContextualCompressionRetriever
from langchain_cohere import CohereRerank

compressor = CohereRerank(model='rerank-english-v3.0', top_n=5)
base_ret = vectorstore.as_retriever(search_kwargs={'k': 20})
rerank_ret = ContextualCompressionRetriever(
    base_compressor=compressor, base_retriever=base_ret
)
docs = rerank_ret.invoke('Explain transformer attention mechanism')
```

02 Query Techniques

The quality of retrieved documents is bounded by the quality of the query. Raw user queries are often ambiguous, under-specified, or phrased differently from the indexed content. Advanced query techniques transform or augment the query to maximize retrieval coverage and precision.

2.1 Query Rewriting

Query rewriting uses an LLM to transform the original user query into a cleaner, more precise, or semantically enriched version before retrieval. It bridges the vocabulary gap between how users ask questions and how documents are written.

Technique	Original Query	Rewritten Query
Clarify ambiguity	Tell me about Python	Explain Python programming language features and use cases
Formalize slang	whats the diff b/w RAM and ROM	What is the difference between RAM and ROM memory?
Expand acronyms	How does RLHF work?	How does Reinforcement Learning from Human Feedback work in LLMs?
Conversational→Standalone	What about its limitations?	What are the limitations of transformer architecture?

HyDE — Hypothetical Document Embedding

HyDE is a powerful rewriting variant: instead of embedding the query directly, an LLM generates a *hypothetical ideal answer* document, which is then embedded for retrieval. Since the generated document uses the same vocabulary and style as real documents, it produces a better query vector than the original question alone.

```
# HyDE with LangChain
from langchain.chains import HypotheticalDocumentEmbedder
from langchain_openai import OpenAI, OpenAIEmbeddings

llm = OpenAI(temperature=0)
embeddings = OpenAIEmbeddings()
hyde_emb = HypotheticalDocumentEmbedder.from_llm(llm, embeddings, multi=False)

# This creates a hypothetical doc and embeds it for retrieval
result = vectorstore.similarity_search_by_vector(
    hyde_emb.embed_query('How does RAG prevent hallucinations?')
)
```

2.2 Multi-Query Retrieval

A single query embedding captures only one semantic perspective. **Multi-query retrieval** generates multiple paraphrased or perspective-shifted versions of the original query, performs retrieval for each, then merges and deduplicates the results. This dramatically improves recall.



Figure 2.1 — Multi-Query Retrieval Pipeline

Example: Query Variants Generated by LLM

Original: How does attention work in transformers?

- Q1: What is the self-attention mechanism in neural networks?
- Q2: Explain scaled dot-product attention in transformers
- Q3: How do transformers learn contextual word representations?
- Q4: What is multi-head attention and why is it used in BERT?

```
from langchain.retrievers.multi_query import MultiQueryRetriever
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(temperature=0)
retriever = MultiQueryRetriever.from_llm(
    retriever=vectorstore.as_retriever(search_kwargs={'k': 5}),
    llm=llm
)
# Automatically generates multiple queries and merges results
docs = retriever.invoke('How does attention work in transformers?')
print(f'Retrieved {len(docs)} unique docs from multi-query')
```

2.3 Query Expansion

Query expansion broadens the query's semantic scope by adding synonyms, related terms, domain keywords, or context. Unlike rewriting (which replaces), expansion *augments* the original query. It is especially effective for specialized domains (medical, legal, financial) where terminology varies.

Method	How It Works	Pros / Cons
Synonym Expansion (WordNet)	Add synonyms from lexical database	Fast; domain-agnostic May add noise for technical terms
LLM-based Expansion	LLM generates related concepts and sub-questions	High quality; domain-aware Adds latency per call
PRF (Pseudo-Relevance Feedback)	Expand using top-K initially retrieved doc terms (BM25/TF-IDF)	No LLM needed Propagates retrieval errors
Step-Back Prompting	Ask a higher-level abstraction question first	Better for reasoning tasks Requires prompt engineering

03 Graph RAG ★ Important

Graph RAG extends traditional vector-based RAG by representing knowledge as a structured graph of entities and their relationships. While vector RAG excels at finding semantically similar chunks, **Graph RAG enables multi-hop reasoning**, relationship traversal, and structured fact extraction that dense retrieval alone cannot achieve.

Why Graph RAG Outperforms Vector RAG for Complex Queries

Vector RAG Limitation • Cannot answer: "What are all the subsidiaries of Company X and which ones are in litigation?" • Chunks are isolated — no structural relationships • Multi-hop queries require multiple retrieval calls • No entity disambiguation (Apple Inc. vs apple fruit)	Graph RAG Advantage • Traverses entity relationships in one query • Explicit edges encode semantic relationships • Supports Cypher/SPARQL for precise fact retrieval • Entities are nodes — no disambiguation issue
--	--

3.1 Knowledge Graph-Based Retrieval

A **knowledge graph (KG)** is a directed labeled graph $G = (V, E, R)$ where V = entities (nodes), E = relationships (edges), and R = relationship types. In Graph RAG, this graph is built from source documents using NLP entity extraction and stored in a graph database (Neo4j, Amazon Neptune, TigerGraph) alongside — or instead of — a vector store.

Graph RAG Architecture



Figure 3.1 — Graph RAG Pipeline

Two Retrieval Modes in Graph RAG

Mode	Mechanism	Best For
Local Search	Entity-anchored: find all nodes connected to query entity within N hops	Specific entity facts, relationship chains
Global Search	Community detection: summarize clusters of highly connected entity groups	Thematic analysis, high-level summaries
Hybrid (Local + Global)	Combine entity traversal with community summary for multi-faceted answers	Complex analytical questions

Microsoft GraphRAG Framework

Microsoft's open-source GraphRAG (2024) uses LLMs to automatically extract entities and relationships from large corpora, builds a hierarchical community structure using the Leiden algorithm, and generates community summaries. At query time, it routes between local and global search based on question type.


```
# Microsoft GraphRAG – Index and Query
pip install graphrag

# Initialize workspace
python -m graphrag.index --init --root ./my_project

# Build graph from documents in ./input/
python -m graphrag.index --root ./my_project

# Local search (entity-specific)
python -m graphrag.query --root ./my_project \
    --method local 'What did Elon Musk say about AI regulation?'

# Global search (thematic)
python -m graphrag.query --root ./my_project \
    --method global 'What are the main themes in AI policy discussions?'
```

LlamaIndex Property Graph + Neo4j

```
from llama_index.core import PropertyGraphIndex
from llama_index.graph_stores.neo4j import Neo4jPropertyGraphStore

graph_store = Neo4jPropertyGraphStore(
    username='neo4j', password='password',
    url='bolt://localhost:7687'
)
index = PropertyGraphIndex.from_documents(
    documents,
    property_graph_store=graph_store,
    show_progress=True
)
query_engine = index.as_query_engine(include_text=True)
response = query_engine.query('Who founded OpenAI and what did they do next?')
```

3.2 Entity Relationships

Entities are the nodes of the knowledge graph; relationships are the typed, directed edges. Together they encode structured factual knowledge that enables precise, verifiable retrieval without semantic ambiguity.

Entity Types and Relationship Categories

Entity Type	Examples	Typical Relations
Person	Elon Musk, Sam Altman	FOUNDED, WORKS_AT, KNOWS, ACQUIRED
Organization	OpenAI, Google DeepMind	ACQUIRED, PARTNERED_WITH, COMPETES_WITH
Technology	Transformer, RLHF, RAG	BASED_ON, INVENTED_BY, USED_IN
Location	San Francisco, Berlin	LOCATED_IN, HEADQUARTERED_AT
Event	NeurIPS 2023, GPT-4 launch	OCCURRED_ON, ATTENDED_BY, PRECEDED_BY

Concept	Attention Mechanism, Fine-tuning	IS_A, PART_OF, REQUIRES
---------	----------------------------------	-------------------------

Cypher Query Examples (Neo4j)

```
// Find all people who founded organizations in AI
MATCH (p:Person)-[:FOUNDED]->(o:Organization {domain: 'AI'})
RETURN p.name, o.name, o.founded_year

// Multi-hop: Who are colleagues of people who worked at OpenAI?
MATCH (p1:Person)-[:WORKS_AT]->(OpenAI)
    -[:WORKS_AT]->(p2:Person)
WHERE OpenAI.name = 'OpenAI' AND p1 <> p2
RETURN DISTINCT p2.name

// Find technology dependencies (3 hops)
MATCH path=(t:Technology {name:'GPT-4'})-[:BASED_ON*1..3]->(base)
RETURN path
```

Entity Extraction with LLMs (LangChain + LLMGraphTransformer)

```
from langchain_experimental.graph_transformers import LLMGraphTransformer
from langchain_openai import ChatOpenAI
from langchain_community.graphs import Neo4jGraph

llm = ChatOpenAI(temperature=0, model='gpt-4o')
graph = Neo4jGraph(url='bolt://localhost:7687',
                  username='neo4j', password='password')
transform = LLMGraphTransformer(llm=llm)

# Convert documents to graph nodes + edges
graph_docs = transform.convert_to_graph_documents(documents)
graph.add_graph_documents(graph_docs, baseEntityLabel=True,
                          include_source=True)

# Query the graph
result = graph.query(
    'MATCH (n:Person)-[r]->(m) RETURN n.id, type(r), m.id LIMIT 10'
)
```

Graph RAG vs Vector RAG — Decision Guide

Criterion	Use Vector RAG	Use Graph RAG
Query type	Semantic similarity, open Q&A	Structured facts, relationships
Data structure	Unstructured prose, documents	Entities with clear relationships
Multi-hop needed?	No	Yes — traverse N hops
Explainability	Low (similarity scores)	High (explicit reasoning paths)
Setup complexity	Low	High (graph DB + schema)

Best tool	FAISS, Chroma, Weaviate	Neo4j + GraphRAG / LlamaIndex
-----------	-------------------------	-------------------------------

04 Multi-Modal RAG

Multi-Modal RAG extends retrieval pipelines beyond plain text to handle **PDFs, images, tables, charts, audio, and video**. Enterprise knowledge bases rarely contain pure text — they include scanned documents, slide decks, technical diagrams, and mixed-format reports. Multi-modal RAG unlocks retrieval over this full spectrum of enterprise content.

4.1 Text + PDF + Image Ingestion

Ingestion Pipeline Overview

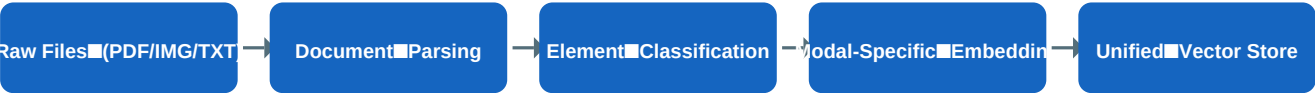


Figure 4.1 — Multi-Modal Ingestion Pipeline

PDF Ingestion Strategies

Tool / Library	Capability	Best For
PyMuPDF (fitz)	Text, images, tables, metadata extraction	General PDFs, scanned docs
pdfplumber	Precise table and text layout extraction	Financial/tabular PDFs
Unstructured.io	Partitions text, tables, images, headers	Enterprise mixed-format PDFs
LlamaParse	LLM-powered PDF parsing with layout awareness	Complex multi-column PDFs
Azure Doc Intelligence	OCR + form recognition + table extraction	Scanned / handwritten docs
AWS Textract	OCR, form fields, table cells from images	High-volume document processing

Image Ingestion Strategies

Images in documents (charts, diagrams, screenshots, photos) require special treatment. There are three main strategies for making image content retrievable:

Strategy	Mechanism	Embedding Type
Image Captioning (LLM-based)	GPT-4V / Claude 3 / Gemini generates a text description of the image	Caption → text embedding (OpenAI, Cohere)
CLIP Embeddings (Vision-Language)	Shared embedding space for image and text (OpenAI CLIP, BLIP-2)	Image embedding stored alongside text embeddings
OCR + NLP	Extract text from image using Tesseract / Azure Vision / Textract	Extracted text → text embedding
Multimodal LLM (end-to-end)	Pass image directly to multimodal LLM (GPT-4V, Claude Vision)	No separate embedding; LLM processes directly

Multi-Modal Ingestion with Unstructured.io

```
from unstructured.partition.auto import partition
from unstructured.chunking.title import chunk_by_title

# Partition any file: PDF, DOCX, PPTX, HTML, image, etc.
elements = partition(filename='annual_report_2024.pdf',
                     strategy='hi_res',          # OCR-enabled
                     extract_images_in_pdf=True)

# Separate by element type
from unstructured.documents.elements import Table, Image, NarrativeText
tables = [e for e in elements if isinstance(e, Table)]
images = [e for e in elements if isinstance(e, Image)]
text   = [e for e in elements if isinstance(e, NarrativeText)]

# Chunk text sections
chunks = chunk_by_title(elements, max_characters=1000)
```

Multi-Modal RAG with LlamaIndex (ColPali / GPT-4V)

```
from llama_index.core import SimpleDirectoryReader, VectorStoreIndex
from llama_index.multi_modal_llms.openai import OpenAIMultiModal
from llama_index.core.indices.multi_modal import MultiModalVectorStoreIndex

# Load mixed documents (text + images)
documents = SimpleDirectoryReader('./data',
                                  recursive=True).load_data()

# Build multi-modal index
mm_llm = OpenAIMultiModal(model='gpt-4o', max_new_tokens=1500)
mm_index = MultiModalVectorStoreIndex.from_documents(documents)

# Query across text and images
retriever = mm_index.as_retriever(similarity_top_k=5,
                                  image_similarity_top_k=3)
response = mm_index.as_query_engine(multi_modal_llm=mm_llm)\
    .query('Summarize the revenue chart on page 4')
```

4.2 Structured vs Unstructured Data in Multi-Modal RAG

Enterprise data exists on a spectrum from fully unstructured (free-form text, raw images) to fully structured (relational tables, JSON). Multi-modal RAG systems must handle all points on this spectrum with appropriate parsing, storage, and retrieval strategies.

Unstructured Data

Semi-Structured Data

Structured Data

• Free-form text (emails, reports, articles) • Scanned PDFs (no text layer) • Images, diagrams, photos • Audio transcripts • HTML web pages Approach: Chunk + embed with text/vision encoder	• PDFs with tables and headers • JSON / XML documents • Markdown files • PPTX / DOCX files • HTML with structure Approach: Element detection + type-specific handling	• SQL databases (RDBMS) • CSV / Excel spreadsheets • Parquet / Delta tables • JSON APIs with schema • Knowledge graphs Approach: Text-to-SQL / graph query + RAG fusion
---	--	--

Text-to-SQL for Structured Data RAG

When structured data (SQL, CSV, Parquet) is the source, natural language queries are translated into SQL using an LLM, executed against the database, and results are passed back to the LLM for final answer synthesis — a pattern called **SQL RAG** or **NL2SQL RAG**.

```
from llama_index.core import SQLDatabase, NLSQLTableQueryEngine
from sqlalchemy import create_engine

engine = create_engine('sqlite:///sales.db')
sql_db = SQLDatabase(engine, include_tables=['orders', 'products'])

query_engine = NLSQLTableQueryEngine(sql_database=sql_db)
response = query_engine.query(
    'What were the top 5 products by revenue in Q3 2024?'
)
# Internally: NL → SQL → Execute → LLM synthesizes answer
print(response) # Final natural language answer
```

Hybrid Multi-Modal RAG Architecture

Production-grade Multi-Modal RAG systems combine multiple retrieval paths and route queries to the appropriate handler based on query intent and available data types.

Query Router	Routes to	Storage	Embedding / Parser
Semantic Q&A	Vector Store	Chroma / Weaviate / Pinecone	OpenAI text-embedding-3
Image / Chart Q	Image Index	Multi-modal vector store	CLIP / GPT-4V caption
Tabular / Stats Q	SQL Engine	PostgreSQL / DuckDB	LLM Text-to-SQL
Relational / KG Q	Graph DB	Neo4j / Amazon Neptune	Cypher query gen
Document layout Q	Parsed Elements	Unstructured chunks	Unstructured.io + embed

Advanced RAG — Key Takeaways

- **Retrieval Optimization:** Tune Top-K dynamically; always apply cross-encoder reranking for precision-critical applications.
- **Query Techniques:** Combine HyDE + multi-query retrieval for maximum recall; use PRF for domain-specific expansion without LLM overhead.

- **Graph RAG:** Use for multi-hop reasoning and entity-relationship queries; Microsoft GraphRAG + Neo4j is the leading open-source stack.
- **Multi-Modal RAG:** Unstructured.io + LlamaIndex cover 80% of enterprise ingestion needs; route to SQL RAG for structured data.
- **Production tip:** Always implement a query router to direct traffic to the right retrieval path rather than applying all techniques to every query.